

Generative Modelling Case Study

Name : Anil Kumar Chava

Student ID : 24068000

Github : <https://github.com/chavaanilkumar/Generative-modelling-case-study>

Contents

1. Introduction	3
2. Part 1: Building and Understanding GANs from Scratch	4
2.1 Reproducing the Sine-Wave GAN	4
2.2 Modelling a Mixture of Gaussians	4
2.3 Modified GAN Architecture	4
2.4 Discussion	5
3. Part 2.1: Optical Coherence Tomography (OCT) Retinal Image Generation	5
3.1 Dataset and Model Selection	5
3.2 Training and Model Performance	6
3.3 Discussion	6
4. Part 2.2: Cybersecurity Synthetic Traffic Generation Using CICIDS2017	7
4.1 Dataset and Model Selection	7
4.2 Training and Evaluation	8
5. Part 2.3: Creative AI – QuickDraw Birthday Cake Generation	8
5.1 Dataset and DCGAN Implementation	8
5.2 Results and Interpretation	9
6. Conclusion	9
References	10

1. Introduction

Generative Adversarial Networks (GANs) are a type of deep learning that can produce synthetic data that is very similar to real-world data. A GAN is composed of two neural networks: a generator that generates synthetic samples from random noise, and a discriminator that classifies real samples and fake samples. They are trained adversarially together, with the generator gradually making the generated images more realistic (Goodfellow et al., 2014).

The goal of this task was to develop GANs with PyTorch and test them on the synthetic and real datasets. This work was divided into 2 parts. In Part 1, the fundamentals of GANs were introduced by recreating a sine-wave GAN, modelling a new two-dimensional Mixture of Gaussians distribution and changing the network architecture to see the effect it has on the generated samples. Part 2 continued these ideas for three practical applications, specifically generating the image of a retina from an OCTMNIST dataset, generating synthetic cybersecurity traffic from a CICIDS2017 dataset, and generating sketches using the Google QuickDraw Birthday Cake dataset.

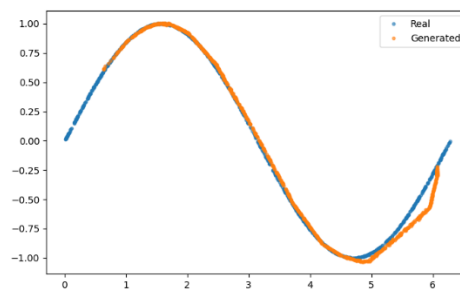
Depending on the type of data, various network architectures were chosen. For low-dimensional synthetic and tabular datasets a fully connected GAN was used and for image generation a Deep Convolutional GAN (DCGANs) was used as convolutional layers are good at learning spatial image features (Radford et al., 2015). Model performance was assessed based on the generator and discriminator loss curves, visual comparisons of the real and generated samples and Principal Component Analysis (PCA) for tabular data and the Fréchet Inception Distance (FID) algorithm for image-based tasks (Pedregosa et al., 2011).

2. Part 1: Building and Understanding GANs from Scratch

2.1 Reproducing the Sine-Wave GAN

Their first task was to recreate the sine-wave GAN that was shown during the tutorial. Two dimensional synthetic data was created with random x values and a sine wave y value. The fully connected GAN architecture was implemented on PyTorch, where the generator was given a random latent vector and then generated 2D coordinates, and the discriminator was able to tell if a sample was real or generated (Paszke et al., 2019). The training was done with the Binary Cross Entropy (BCE) loss and the Adam optimiser. As training continued, the samples produced became more and more sinusoidal. The visual comparison indicated that the generator was able to accurately reflect the nonlinear relationship in the dataset, indicating that the adversarial learning can be used to model simple probability distributions (Ha & Eck, 2017).

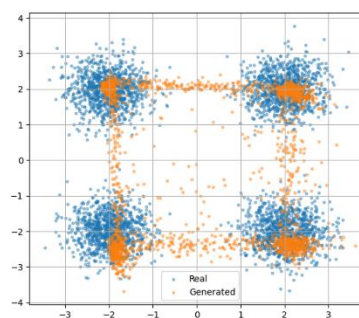
Figure 1: Real vs. Generated Sine-Wave Samples



2.2 Modelling a Mixture of Gaussians

For assessing the GAN with a more complex distribution, a Mixture of Gaussians dataset with multiple clusters was created. Unlike the continuous sine wave pattern, this data set demanded the generator to acquire multiple sets of data in the same time period. The same fully connected GAN architecture was used and trained with the new dataset. The losses of the generator and discriminator gradually converged towards zero during training, suggesting that both networks were learning effectively.

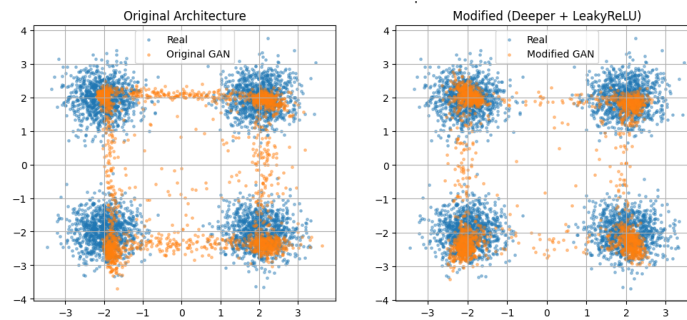
Figure 2: Mixture of Gaussians – Real vs. Generated Samples



2.3 Modified GAN Architecture

The last assignment explored the possibility of enhancing the quality of generation through architectural changes. Two changes were introduced they are additional hidden layers were added to increase model capacity. LeakyReLU was used instead of ReLU activation functions to enhance training in the network. These changes led to more stable training and led to samples that were closer to the original Gaussian distribution. The modified architecture generated fewer scattered points and captured the cluster boundaries in a better manner when compared with the baseline model. This is in line with the findings of Radford et al. (2015).

Figure 3: Comparison of Original and Modified GAN Outputs



2.4 Discussion

In Part 1, the adversarial learning was demonstrated by moving from a sine wave distribution to a multimodal distribution of data. The experiments showed that a GAN can be trained to learn a complex probability distribution without explicit supervision and that relatively small changes in the architecture could make a noticeable improvement in the quality of samples generated.

3. Part 2.1: Optical Coherence Tomography (OCT) Retinal Image Generation

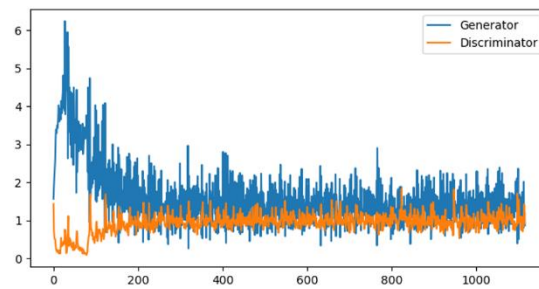
3.1 Dataset and Model Selection

The first real application was based on the OCTMNIST dataset from MedMNIST (Yang et al., 2021). The first step of the exploration was to analyze the distribution of the classes and present some example images of the retina to gain insight into the nature of the data before training the GAN. To make the training more stable, the image pixel values were normalised. Since the images are predominantly used, a Deep Convolutional Generative Adversarial Network (DCGAN) was chosen. Compared to fully connected networks, convolutional layers maintain the spatial context of the image and are better suited to capture image properties like edges, textures, and anatomical structures (Radford et al., 2015). The generator would convert random latent vectors to synthetic retinal images and the discriminator would determine whether the images were real or synthetic.

3.2 Training and Model Performance

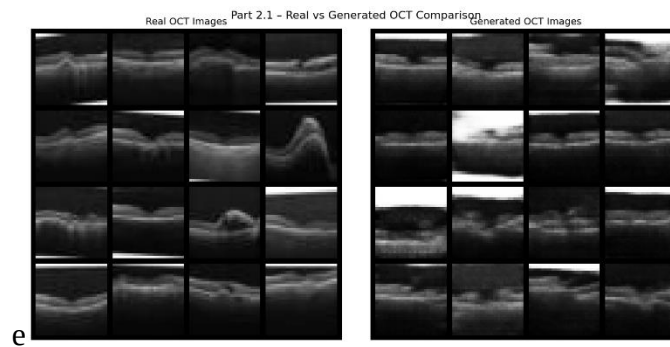
The DCGAN was created with the Adam optimiser and Binary Cross Entropy (BCE) loss function. Losses from the generator and discriminator were measured during training to track convergence. Both losses were dynamic, as would be expected in adversarial learning, as learning in one network hurt performance in the other network for a while. The total training was, however, unaffected and showed that both models continued training through the training process.

Figure 4: Generator and Discriminator Loss Curves



The generator generated synthetic retinal images from previously unseen random noise vectors after training. The images generated by the model were compared with the original OCT images, and it was found that the model was able to learn the overall structure and intensity patterns of the retina.

Figure 5: Real vs. Generated OCT Images



In order to quantify the evaluation, the Fréchet Inception Distance (FID) was determined. The implementation reported $FID = 0.03$, which is a good score, denoting that the feature distribution of the generated image is highly similar to the real images' feature distribution. This is an indicator of good model performance but it doesn't necessarily confirm clinical realism and visual inspection should also be carried out.

3.3 Discussion

The results illustrate the ability of DCGANs to learn the distribution of retinal OCT images and produce synthetic samples that are similar to real ones. These images could then be used to augment medical images, for educational use, or for privacy-protecting research with restricted access to clinical data. Nevertheless, generated images should be used in conjunction with real medical images, since subtle anatomic inaccuracies might have an impact on clinical interpretation. Datasets of duck photos might be used in place of retinal images. If so, the generator would be able to learn the features of the body shape, feathers, and even colour patterns indicating that GANs can be a flexible model capable of synthesizing images in a different domain even with enough training data.

4. Part 2.2: Cybersecurity Synthetic Traffic Generation Using CICIDS2017

4.1 Dataset and Model Selection

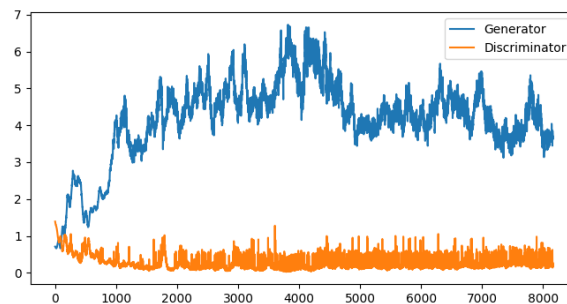
In the second application, the authors used GANs to create synthetic network traffic with the CICIDS2017 network traffic dataset. The data for working hours on Wednesdays (including BENIGN and DoS traffic) was loaded and explored, following the assignment requirements to understand the class distribution and feature characteristics. The data set was composed of numerical network traffic features, not suitable for a convolutional network, but a fully connected GAN is more suitable. The numerical features were subsequently preprocessed and standardised for stability in training (Sharafaldin et al., 2018).

The discriminator was trained to distinguish between true and synthetic network traffic records, and a fully connected generator was designed to produce synthetic network traffic records from random latent vectors. The choice of architecture was made because fully connected layers provide a good representation of relationships in structured tabular data.

4.2 Training and Evaluation

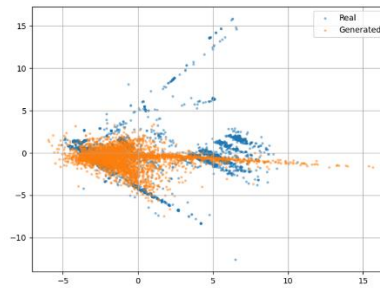
The generator was trained for 30 epochs using the Adam optimiser and losses of both the generator and discriminator were tracked during the training process. The loss curves were expected and neither network ended up at zero but were continually adapting as they were optimized.

Figure 6: Generator and Discriminator Loss Curves



Principal Component Analysis (PCA) was used to reduce the high dimensional feature space to 2 dimensions to be evaluated for the generated traffic. The PCA visualisation revealed a high degree of similarity between the real and generated samples, thus demonstrating that many of the statistical characteristics of the original network traffic were captured by the generator.

Figure 7: PCA Comparison of Real and Generated Traffic



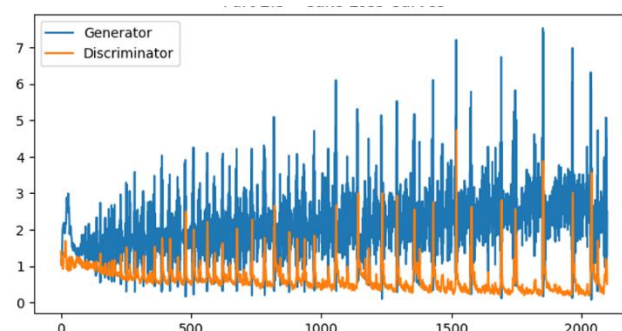
The resulting samples of data were in general distributed similarly to the original data, but there was some separation between the two groups. This indicates that the overall feature patterns were learned by the GAN, but not exactly replicated for all the features. Performance can be further enhanced with additional training, or through the use of more advanced architectures such as Conditional GANs (cGANs) or Wasserstein GANs (WGANs). The adversarial learning method used was also applicable in image generation. For instance, if the cybersecurity data was replaced by pictures of ducks, the generator could be trained on visual characteristics like shape, texture, and color to generate realistic duck images, for example. This is a proof of the versatility of using GANs in various application areas.

5. Part 2.3: Creative AI – QuickDraw Birthday Cake Generation

5.1 Dataset and DCGAN Implementation

The final application was based on the Google QuickDraw Birthday Cake dataset that created hand-drawn Birthday Cake sketches. Prior to training the model, sample sketches were explored to gain an understanding of the drawing style. Since the data was represented as images, a DCGAN architecture was chosen as it is known to be a better architecture for learning spatial features than a fully connected network (Radford et al., 2015). Random latent vectors were fed into the generator to produce sketch images and the discriminator was used to classify sketch images as real or generated. The training employed Binary Cross Entropy loss and the Adam optimiser, and the loss of the generator and discriminator was tracked during training (Waskom, 2021).

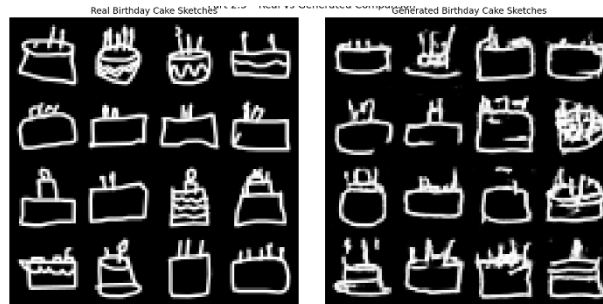
Figure 8: Generator and Discriminator Loss Curves



5.2 Results and Interpretation

The training resulted in generator-generated synthetic birthday cake sketches that were able to capture the overall shape and composition of the original drawings. A majority of the generated sketches had identifiable cake shapes and decorations, but a few instances exhibited slight deformations or sketch outlines that were not fully developed, typical behavior in GAN training on low numbers of epochs.

Figure 9: Real vs. Generated Birthday Cake Sketches



The Fréchet Inception Distance (FID) was also used to assess the quality of the images generated. The implementation reported an FID score of 0.03, with the similarity between the generated and original sketch distributions being good. The numerical result indicates good learning but a visual check is required to ensure that the sketches which are produced retain meaningful shapes and artistry.

Conclusion

This assignment was a showcase of applying and testing Generative Adversarial Networks (GANs) with synthetic and real-world data. The experiments revealed that by adding layers to the GANs, the quality of the generated images could be enhanced, and by adding activation layers to the GANs, such as LeakyReLU, the generated images could be improved to obtain realistic retinal images and birthday cake sketches successfully using the DCGANs. The GAN was trained with the CICIDS2017 dataset and also learned the overall distribution of network traffic as shown in the PCA visualisation (Pedregosa et al., 2011). While the models showed good performance, the stability of the training process and the sensitivity of the models to hyper-parameters still pose challenges for the training of GANs. For future work, Conditional GANs (cGANs), Wasserstein GANs (WGANs) or larger datasets could be investigated to enhance image quality and generalisation.

References

- Goodfellow, I., Pouget-Abadie, J., Mirza, M., Xu, B., Warde-Farley, D., Ozair, S., Courville, A., & Bengio, Y. (2014). *Generative adversarial nets*. *Advances in Neural Information Processing Systems*, 27.
- Google Creative Lab. (2017). *The Quick, Draw! dataset*.
<https://github.com/googlecreativelab/quickdraw-dataset>
- Ha, D., & Eck, D. (2017). *A neural representation of sketch drawings*. arXiv.
<https://arxiv.org/abs/1704.03477>
- Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z., Gimelshein, N., Antiga, L., Desmaison, A., Köpf, A., Yang, E., DeVito, Z., Raison, M., Tejani, A., Chilamkurthy, S., Steiner, B., Fang, L., & Chintala, S. (2019). *PyTorch: An imperative style, high-performance deep learning library*. In H. Wallach et al. (Eds.), *Advances in Neural Information Processing Systems* (Vol. 32). Curran Associates.
- Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, É. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.
- Radford, A., Metz, L., & Chintala, S. (2015). *Unsupervised representation learning with deep convolutional generative adversarial networks*. arXiv.
<https://arxiv.org/abs/1511.06434>
- Sharafaldin, I., Lashkari, A. H., & Ghorbani, A. A. (2018). Toward generating a new intrusion detection dataset and intrusion traffic characterization. *Proceedings of the International Conference on Information Systems Security and Privacy (ICISSP)*, 108–116.
- Waskom, M. L. (2021). Seaborn: Statistical data visualization. *Journal of Open Source Software*, 6(60), 3021. <https://doi.org/10.21105/joss.03021>
- Yang, J., Shi, R., & Ni, B. (2021). MedMNIST classification decathlon: A lightweight AutoML benchmark for medical image analysis. *IEEE 18th International Symposium on Biomedical Imaging (ISBI)*, 191–195.